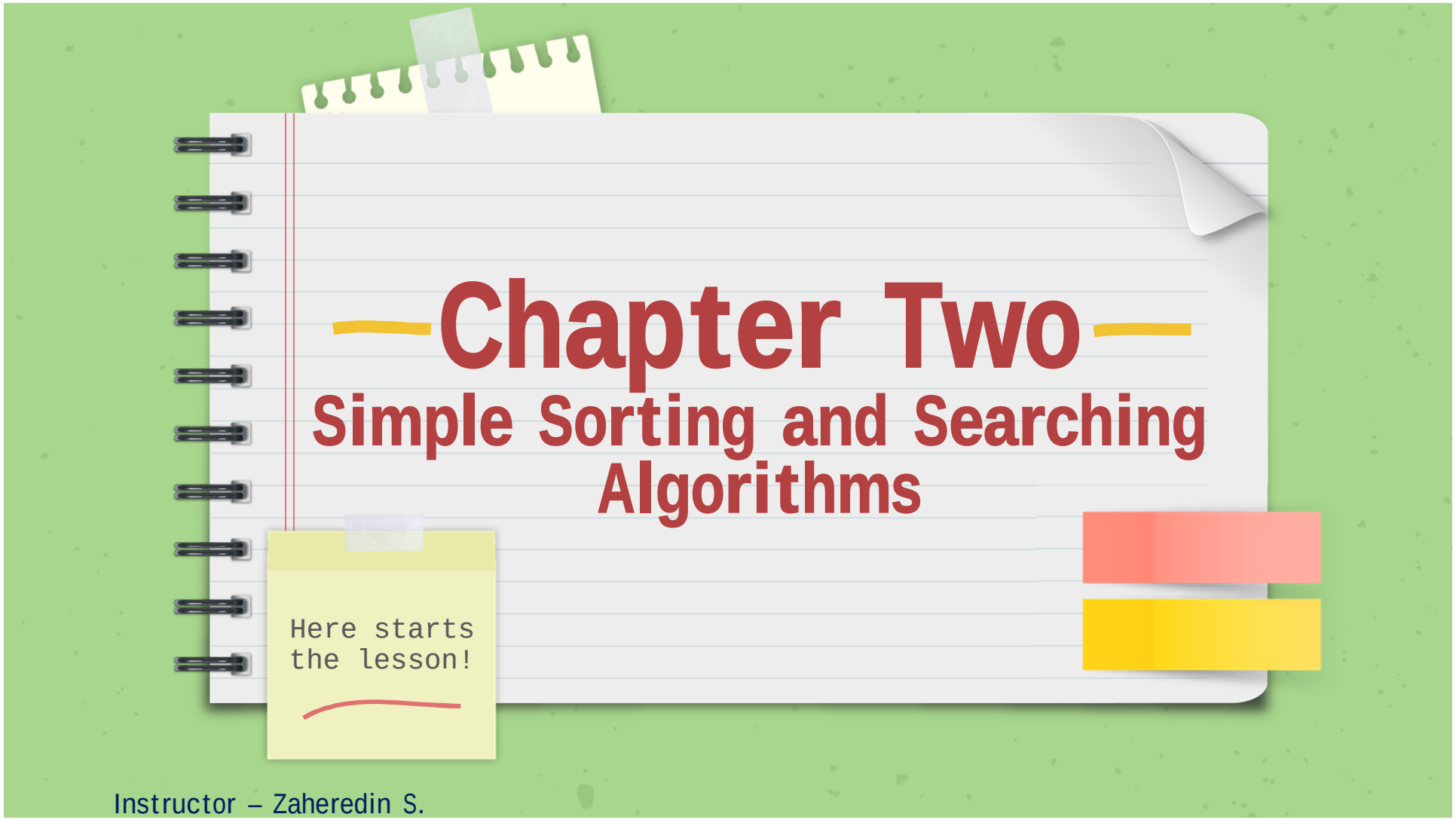


Instructor – Zaheredin S.

Chapter Two Contents!

2.1 Sorting Algorithms

- 2.1.1 Insertion Sort
- 2.1.2 Selection Sort
- 2.1.3 Bubble Sort

Chapter Two Contents!

2.2 Searching Algorithms

- 2.2.1 Linear Search
- 2.2.2 Binary Search

2.1 – Sorting Algorithms

- **Sorting** is one of the most important operations performed by computers.
- Sorting is a process of reordering (re arranging) a list of items (elements) in to either ascending or descending order within the array.
- If the elements are in character type ordered by alphabetical else numbers ordered by 1,2,3,4....or 5,4,3,2,1...

Example:- 7,39,8,45,35,29

Ascending order (7,8,29,35,39,45)

Descending order (45,39,35,29,8,7)

Cont...

- During the sorting process the data traversed many times.
- Each traversal of the data is referend as a Sort Pass.

Cont...

Importance of Sorting in Data Structure

- When you are performing sorting on elements, many complications such as min/max, kth smallest/largest get automatically simplified.
- Sorting provides you with many algorithmic solutions, which might include divide and conquer, iterative, and recursive based.
- Last but not least, perhaps one of its biggest benefit is time complexity. It not only saves up your very precious time but also provides you with right solution.

Cont...

Types of sorting

- Insertion Sort
- Selection Sort
- Bubble Sort etc...

Cont...

I. Insertion Sort

- It is a simple sorting algorithm that places an unsorted element at its suitable place in each iteration.
- The array is virtually split into a sorted and unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.
- It inserts each item into its proper place in the final list.
- The simplest implementation requires two list structures, the source list and the list into which sorted items are inserted.

Cont...

- To save memory, most implementations use an in-place sort that works by moving the current item past the already sorted items and repeatedly swapping it with the preceding item until it is in place.

Basic Idea:

- Find the location for an element and move all others up, and insert the element.

The process involved in insertion sort is as follows:

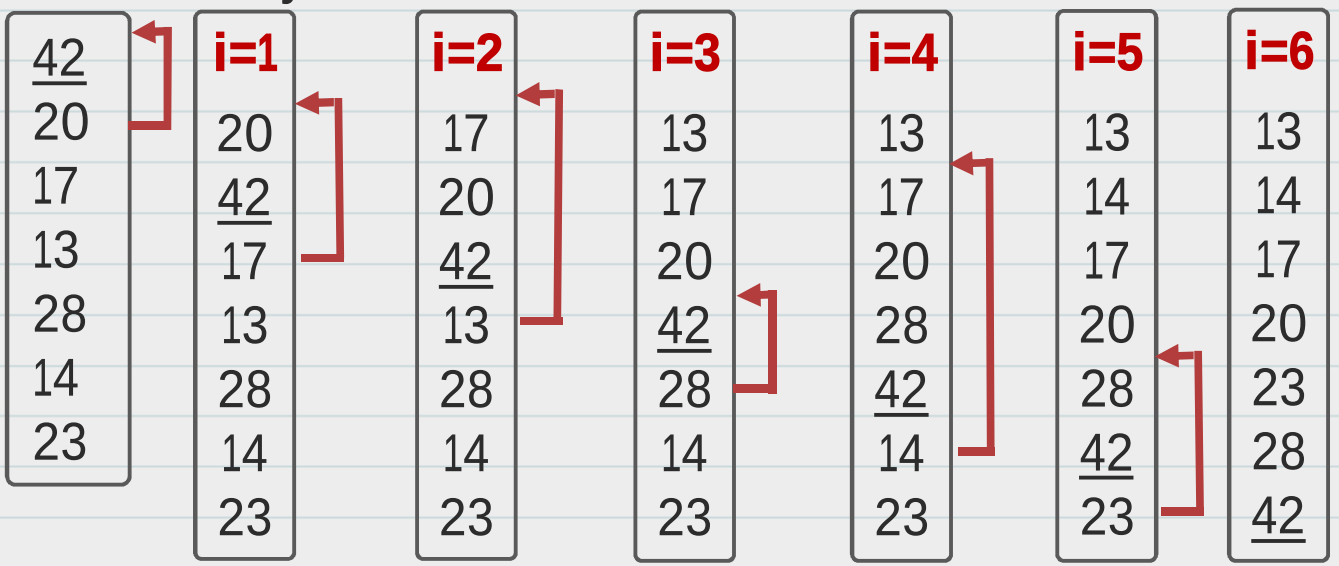
- The left most value can be said to be sorted relative to itself. Thus, we don't need to do anything.

Cont...

- Check to see if the second value is smaller than the first one. If it is, swap these two values. The first two values are now relatively sorted.
- Next, we need to insert the third value in to the relatively sorted portion so that after insertion, the portion will still be relatively sorted.
- Remove the third value first. Slide the second value to make room for insertion. Insert the value in the appropriate position. Now the first three are relatively sorted.
- Do the same for the remaining items in the list.

Cont...

Examples of inserting an element into its correct location in the array



Cont...

Pseudocode for the insertion sort algorithm is given below:

```
InsertionSort(data):
    for each element i in data, starting from the leftmost position:
        shift all previous elements that are greater than data[i] right one position
        place data[i] in its correct position
```

Cont...

Implementation

```
void insertion_sort(int list[]){
    int temp;
    for(int i=1 ; i<n ; i++){
        temp=list[i];
        for(int j=i; j>0 && temp<list[j-1];j--){
            list[j]=list[j-1];
            list[j-1]=temp;
        }
    }
}
```

Cont...

C++ Program to Implement Insertion Sort

```
#include <iostream>
using namespace std;
int main() {
    int a[5]={10, 5, 91, 11, 8};
    int i, j, k, temp;
    for (i = 1; i < 5; i++)
        for (j = i; j >= 1; j--){
            if (a[j] < a[j-1]){
                temp = a[j];   a[j] = a[j-1];   a[j-1] = temp;
            }
        }
}
```

Cont...

Time Complexity for Insertion sort

- Worst Case Complexity: $O(n^2)$
  - It occurs, when array is given in ascending order, and you want to sort it in descending order or vice versa.
- Best Case Complexity: $O(n)$
  - It occurs, when the array is already sorted.
- Average Case Complexity: $O(n^2)$
  - It occurs when the elements of an array are in neither ascending nor descending.

Cont...

Insertion sort Applications

It can be used when

- The array has small number of elements.
- There are only a few elements left to be sorted.

Cont...

II. Selection Sort

- It is a sorting algorithm that selects the smallest element from unsorted list in each iteration and places that element at the beginning of the unsorted list.
- First, the array is scanned to find the smallest element.
- This is swapped with the element in the first position.
- Next, the remainder of the array is scanned to find the next smallest element.
- This is swapped with the element in the second position.

Cont...

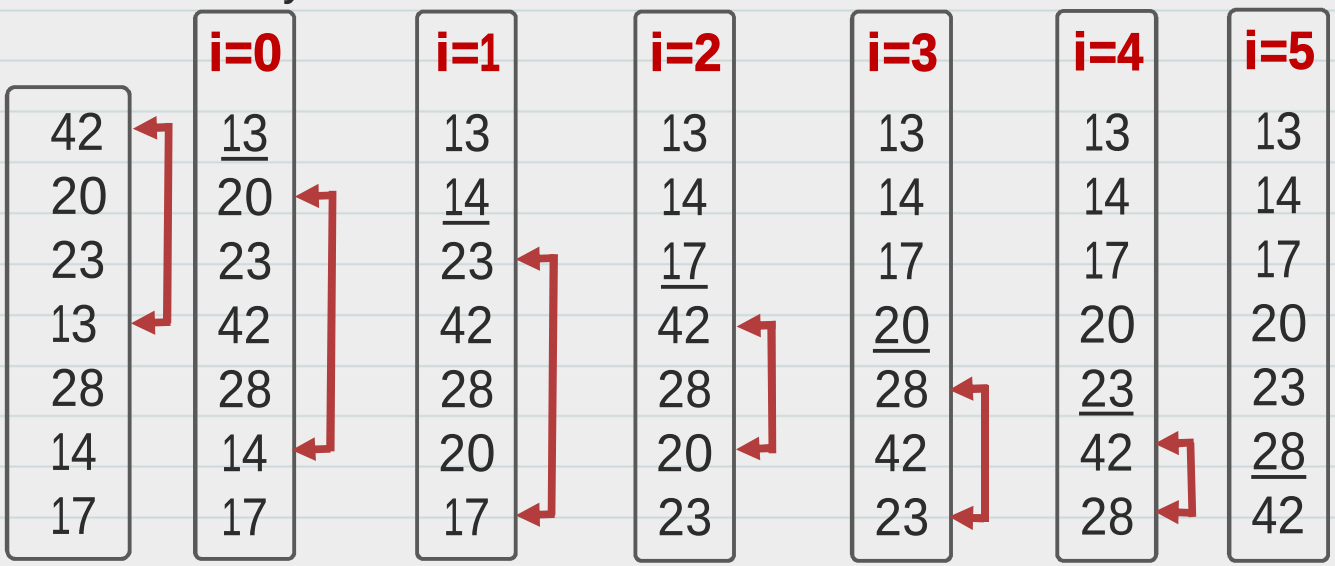
- This process continues until all elements have been placed in their correct position.

Pseudocode for the Selection sort algorithm is given below:

```
SelectionSort(data):
    for each element i in data apart from the last
        Select the smallest element from data[i]...data[n-1]
        Swap it with data[i]
```

Cont...

Examples of Selection an element into its correct location in the array



Cont...

Implementation

```
void selection_sort(int list[]){
        for(int i=0; i<n-1; i++)
            int min=i;
            for(int j=i+1; j<n; j++){
                if(list[j]<list[min])
                    min=j;
            }
            temp=list[min];
            list[min]=list[i];
            list[i]=temp;
}
```

Cont...

C++ Program to Implement Selection Sort

```
#include <iostream>
using namespace std;
int main() {
    int min, a[6]={10, 5, 91, 11, 8, 4};
    for (int i = 0; i < 5; i++)
       min = i;
       for (int j = i + 1; j < 6; j++){
          if (a[j] < a[min])
             min = j;
       }
}
```

Cont...

```
        int  temp;
        temp=a[i];
        a[i]=a[min];
        a[min]=temp;
        }
    }
```

Cont...

Time Complexity for Selection sort

- Worst Case Complexity: $O(n^2)$
  - It occurs, when array is given in ascending order, and you want to sort it in descending order or vice versa.
- Best Case Complexity: $O(n^2)$
  - It occurs, when the array is already sorted.
- Average Case Complexity: $O(n^2)$
  - It occurs when the elements of an array are in neither ascending nor descending.

Cont...

Selection Sort Applications

It can be used when

- A small list is to be sorted.
- Cost of swapping does not matter.
- Checking of all the element is compulsory.
- Cost of writing to a memory matters like in flash memory (number of writes/swaps is O(n) as compared to O(n2) on bubble sort.

Cont...

III. Bubble Sort

- Bubble sort is the simplest algorithm to implement and the slowest algorithm on very large inputs.
- It works by repeatedly swapping the adjacent elements if they are in wrong order.
- It starts from the first element. If the elements are in correct order, there is no swapping condition. So, shift to the next index, else the elements put in not correct order swap them.

Cont...

Basic Idea:

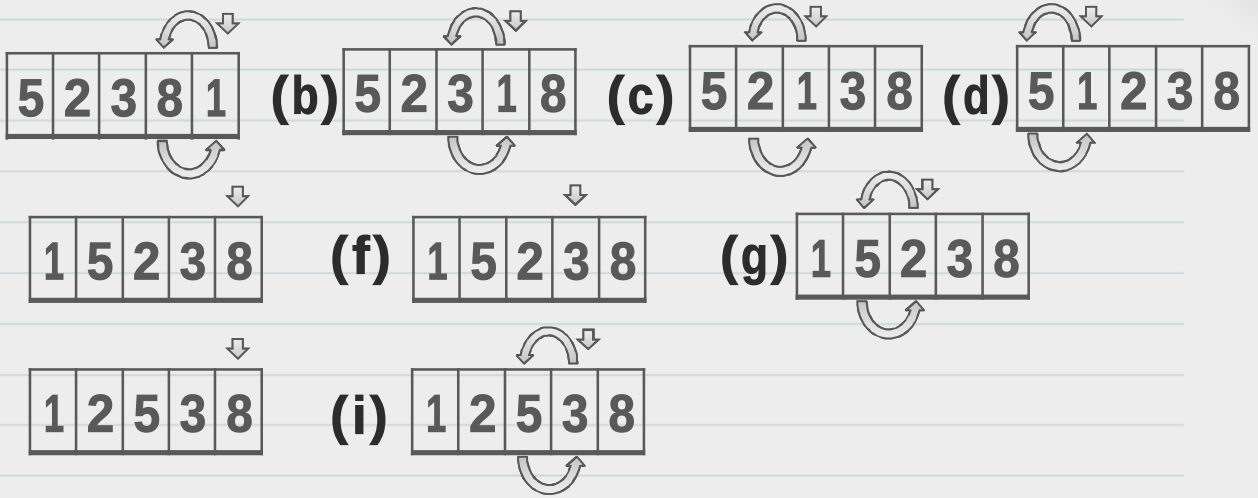
- Loop through array from i=0 to n and swap adjacent elements if they are out of order.

Pseudocode and a C++ implementation for bubble sort are given below:

```
BubbleSort(data):
    for (i=0; i<n-1; i++)
        for (j=0; j<n-i-1; j++)
            Swap elements in positions i and j if out of order
```

Cont...

Examples of Bubble an element into its correct location in the array



Cont...

Implementation

```
void bubble_sort(list[]) {
    for(int i=0;i<n-1; i++){
        for(int j=0;j<n-i-1; j++){
            if(list[j]>list[j+1){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
}
```

Cont...

C++ Program to Implement Bubble Sort

```
#include <iostream>
int main() {
    int temp, a[6]={10, 5, 91, 11, 8, 4};
    for(int i = 0; i < 5; i++)
       for(int j=0; j<6-i-1; j++){
          if (a[j] < a[j+1]){
                    temp = a[j];
                    a[j] = a[j+1];
                    a[j+1] = temp;
          }
       }
```

Cont...

```
        std::cout<<"The bubble sorted array :"<<endl;
        for(int i=0; i<6; i++) {
            cout<<a[i]<<"  ";
        }
}
```

OUTPUT

```
The bubble sorted array :
4  5  8  10  11  91
```

Cont...

Time Complexity for Bubble sort

- ❖ Worst Case Complexity: $O(n^2)$
  - It occurs, when array is given in ascending order, and you want to sort it in descending order or vice versa.
- ❖ Best Case Complexity: $O(n)$
  - It occurs, when the array is already sorted.
- ❖ Average Case Complexity: $O(n^2)$
  - It occurs when the elements of an array are in neither ascending nor descending.

Cont...

Bubble Sort Applications

It can be used when

- Complexity does not matter.
- Short and simple code is preferred.

2.2 – Simple Searching Algorithms

- **Searching** is a process of looking for a specific element in a list of items (array) or determining that the item is not in the list.

There are two simple searching algorithms:

- **Linear Search** (Sequential Search), and
- **Binary Search**

Cont...

I. Linear Search (Sequential Search)

- The linear search is the easiest searching algorithm that starts at one end, and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.

Pseudocode

- Loop through the array starting at the first element until the value of target matches one of the array elements.
- If a match is not found, return –1.

Cont...

Examples of Linear Search **N = 8**

15	5	20	35	2	42	67	17
0	1	2	3	4	5	6	7

Have two conditions;

1. if elements are present in the array, print location (at what index, position)
2. if elements are not present in the array, (print element not found) access each (one by one) from the first (start) element to end.

Cont...

Element 42 has to be searched in it using linear Search Algorithm.

Find key=42

- Present (found the element, then stop when searching algorithm is present in the array.
- **Key 42** (found index 5 (a[5]), position 6))

Find key= 41

- Not Present (searching until end of array and then stop searching. And print key is not present in the array)

Cont...

Implementation

```
int Linear_Search(int list[], int key){
    for(int i=0; i<n; i++){
        if(list[i]==key){
            return i;
        }
        return -1;
    }
}
```

Cont...

Time Complexity for Linear Search

- Time is proportional to the size of input (n) and
- We call this time complexity O(n)

Linear Search Applications

It can be used when

- For searching operations in smaller arrays (<100 items).

Cont...

II. Binary Search

- This algorithm locates specific items by comparing the middlemost items in the data collection.
- This algorithms works only on an ordered (sorted) list.

The basic idea is:

1. Locate midpoint of array to search
2. Determine if target is in lower half or upper half of an array.
   - If in lower half, make this half the array to search
   - If in upper half, make this half the array to search

Cont...

3. Loop back to step 1 until the size of the array to search is one, and this element does not match, in which case return –1.

Cont...

Explanation - The following three cases are possible-

Case-01

- If the element being searched is found to be the middle most element, its index is returned.

Case-02

- If the element being searched is found to be greater than the middle most element, then its search is further continued in the right sub array of the middle most element.

Cont...

Case-03

- If the element being searched is found to be smaller than the middle most element, then its search is further continued in the left sub array of the middle most element.
- This iteration keeps on repeating on the sub arrays until the desired element is found or size of the sub array reduces to zero.

Cont...

Example – From sorted array below, search the location of value 31 using binary search.

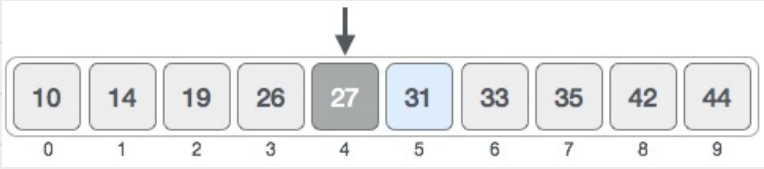


Solution –

- First, Determine half of the array using the formula -

mid = (low + high) / 2

i.e, (0 + 9) / 2 = 4 (integer value of 4.5). So, 4 is the mid of the array.



Cont...

- Compare the searched value, i.e 31 with the value stored at location 4. we know that the target value lie at the right subarray, because 31 is greater than 27.



- We change our low to mid + 1 and find the new mid value again.

```
low = mid + 1
mid = (low + high) / 2
```

- Here, our new mid value is 7. We compare the value stored at location 7 with our target value 31.

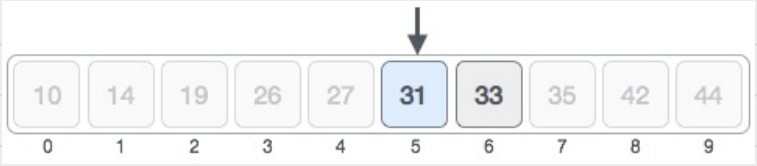
Cont...



- Compare the searched value, i.e 31 with the value stored at location 7. we know that the target value lie at the left subarray, because 31 is less than 35.



- We change our high to (mid – 1) & calculate mid again. This time it is 5.



Cont...

- We compare the value stored at location 5 with our target value. We find that it is a match.



- We conclude that the target value 31 is stored at location 5.

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Cont...

Implementation

```
int Binary_Search(int list[],int k) {
    int left=0, right=n-1, found=0;
    do{
        mid=(left+right)/2;
        if(key==list[mid])
            found=1;
        else if(key<list[mid])
            right=mid-1;
        else
            left=mid+1;
    }while(found==0&&left<=right);
    if(found==0)
        index=-1;
    else
        index=mid;
return index;
}
```

Cont...

Time Complexity for Binary Search

- **Best case complexity:** O(1)
- **Average case complexity:** O(log n)
- **Worst case complexity:** O(log n)


END
Thank You!